Clustbench: A Reproducible, Trajectory-Instrumented, Self-Extending Benchmark for Clustering Algorithms

Ashail Maharaj
https://github.com/Ashail33/clustbench

October 2026

Abstract

Clustering algorithms are hard to compare. Published evaluations differ in datasets, sample sizes, metrics, hyperparameters and hardware, and few report runtime or memory in a reproducible way. We present *Clustbench*, an open-source benchmark harness that makes the comparison mechanical. A YAML configuration expands into a Cartesian grid of tasks over synthetic, real-world, time-series, graph and adversarial datasets with known ground truth. Every registered algorithm, whether native Python or any executable speaking a four-field JSON protocol, runs on the same data, and each run is recorded as one schema-validated row holding external validity (ARI, NMI), internal validity (silhouette, Davies–Bouldin, Dunn) and resource use (wall time, memory delta, CPU and I/O). Results are aggregated by average ranks and a Friedman test. Clustbench adds three things that existing clustering benchmarks do not provide together. First, a *trajectory layer* logs every step of an iterative algorithm as a $(\text{state}, \text{action}, \text{cost}, \Delta\text{cost})$ record, so the optimisation process becomes data. Second, machine-readable *algorithm cards* give theoretical cost and quality predictions next to the measured values. Third, the benchmark's own output is the training set for a *self-extending* layer of learned algorithm selectors that retrain automatically when new algorithms or datasets are added. On the current release (47 algorithms, 19 dataset families, 72 tasks, 3,354 result rows) the Friedman test rejects equal performance decisively ($\chi^2 = 1189$, $p \approx 7 \times 10^{-219}$). No single algorithm wins every dataset family. A separate out-of-distribution benchmark shows that the selectors that rank highest in-distribution lose 0.11–0.13 ARI on unseen regimes, which is the main methodological lesson the harness surfaced.

1 Introduction

Clustering has no single correct objective [1], so the practical question *which algorithm should I use on this data?* is answered empirically. The empirical literature is fragmented. Each paper picks its own datasets, scale, hyperparameter budget, metrics and hardware, and wall time and memory are usually reported anecdotally if at all. A white paper on benchmarking in cluster analysis [2] names these problems and calls for neutral, reproducible comparisons.

The author's MEng research assignment [3] reviewed big-data clustering methods, proposed a four-way taxonomy (sampling/partitioning, ensemble, parallel/distributed and incremental methods), and benchmarked one representative of each category: S^5C [4], PWCC [5], MapReduce-style parallel k-means [6] and mini-batch k-means [7]. Its closing recommendation was a *centralised database for clustering research* that would let practitioners find the right model for their data and infrastructure. That study's experiments were limited by compute: the full factorial grid exceeded the available notebook quota. Clustbench turns the recommendation into software and keeps the grid cheap to re-run.

Contributions.

1. **A language-agnostic harness** (§3) that expands a declarative configuration into a task grid, runs any number of algorithms on identical data, and writes one tidy, schema-validated record per run together with resource measurements, average ranks and a Friedman test.
2. **A trajectory layer** (§3.5) that records the step-by-step optimisation of iterative algorithms as state–action–reward tuples. To our knowledge this is the first clustering benchmark to treat the *search process* as a first-class output.
3. **Algorithm cards, a gap finder and a mutator** (§3.6) that encode each algorithm's mechanism, inductive biases and complexity in machine-readable form, predict cost and quality from them, and search the fingerprint space for regimes no registered algorithm covers.
4. **A benchmark suite** (§4) of more than thirty dataset generators across convex, non-convex, heterogeneous-density, outlier-heavy, low-rank high-dimensional, real tabular, time-series, graph and deliberately adversarial regimes, and a registry of 50 algorithm implementations.
5. **An empirical study** (§5) that includes in-distribution, held-out-seed and out-of-distribution evaluations. It documents a failure mode of learned algorithm selection that only the out-of-distribution split exposes.

A companion paper [8] describes the new algorithms developed with the harness. This paper covers the harness, the suite and what the benchmark measures.

2 Related work

Clustering benchmarks. Van Mechelen et al. [2] set out principles for neutral benchmarking in cluster analysis. Gagolewski [9] proposed a framework and curated battery of low-dimensional benchmark datasets with reference partitions, distributed as a Python package that is *also* named `clustbench`. The two projects are independent and complementary: Gagolewski's work is a dataset battery with reference labels, while ours is a run harness with resource measurement, trajectory capture and learned selection. Gagolewski's batteries could be loaded through our dataset interface. Fahad et al. [10] gave an empirical taxonomy of big-data clustering algorithms that the author's review built on [3].

Synthetic data with ground truth. MDCGen [11] generates multidimensional clustering datasets with controllable outliers, noise and density. Clustbench reimplements an MDCGen-style generator and adds shape generators (moons, circles, spirals, Swiss roll, rings), heavy-tailed and imbalanced mixtures, and adversarial families.

Statistical comparison. We follow Demšar's recommendation [12] of average ranks with the Friedman test [13] for comparing many algorithms over many datasets.

Algorithm selection and meta-learning. Choosing an algorithm from dataset features is Rice's algorithm selection problem [14]. It has been studied as meta-learning in general [15, 16] and for clustering specifically [17, 18], and is used as warm-starting in AutoML [19]. Clustbench's learned routers are instances of this idea. The contribution here is the infrastructure that makes them self-extending: the router training set *is* the benchmark output, so it grows whenever the benchmark runs.

3 Design

3.1 Task grid

A benchmark is a YAML file. Each dataset entry lists value sets for sample size n, dimension d, target cluster count k, compactness and generator-specific knobs (outlier count, noise points,

density, outlier extremity). The harness takes the Cartesian product of every list and crosses it with a list of seeds:

```
datasets:
  - id: mdcgen
    n_samples: [500]
    n_features: [10]
    k_targets: [3, 5]
    outliers: [0, 100]
algorithms:
  - {name: kmeans, kind: python, entry: kmeans, params: {n_init: 3}}
  - {name: mine, kind: external, entry: ./bin/run_my_algo}
seeds: [1, 2, 3]
```

A task is uniquely identified by (dataset, n, d, k, compactness, outliers, noise, density, seed). Every algorithm sees the identical array for a given task.

3.2 Algorithm interface

Native algorithms subclass a small abstract base class, register themselves under a string key, and implement `fit_predict(X, k)`, which returns an `AlgoResult` with integer labels, an optional dictionary of algorithm-specific extras, and an optional trajectory. External algorithms in any language are supported through a JSON-over-stdin protocol. The harness writes the task to a `.npy` file and sends

```
{"data_path": "...", "k": 5, "params": {...},
 "artifacts_dir": "...", "labels_path": "..."}
```

on standard input. The executable writes integer labels to `labels_path` and may print a JSON object of extras to standard output. That is the entire protocol.

3.3 Records and resource measurement

Each (algorithm, task) pair produces one record validated by a Pydantic schema (Appendix A). Resource use is measured with `psutil` around the `fit_predict` call: wall-clock time, resident-set-size delta, user and system CPU time, and read/write bytes where the platform exposes them. Metrics are computed by one function shared by all algorithms: ARI [20] and NMI [21] against ground truth; silhouette [22] (sub-sampled above 5,000 points), Davies–Bouldin [23] and Dunn [24] as internal indices; plus compactness and separation. The results table is flat (one column per metric), so a leaderboard is a one-line query in pandas or DuckDB. A crashing algorithm is logged and skipped and does not abort the sweep. In the 4,332-row comprehensive run, 156 (3.6%) runs failed this way and every other result was kept.

3.4 Aggregation

For each metric, algorithms are ranked within every task (rank 1 = best, ties averaged). Ranks are averaged across tasks, and the Friedman statistic is computed over the tasks on which every algorithm produced a result [12]. The summary is written to `summary.json` and also recomputed in the browser by the dashboard, so adding an algorithm never requires changing analysis code.

3.5 The trajectory layer

Iterative clustering algorithms are sequential decision processes. At each step they observe a state (centroids, medoids, an assignment, a mixture's parameters), take an action (reassign, move a centroid, swap a medoid, merge two clusters), and observe a change in objective. Clustbench

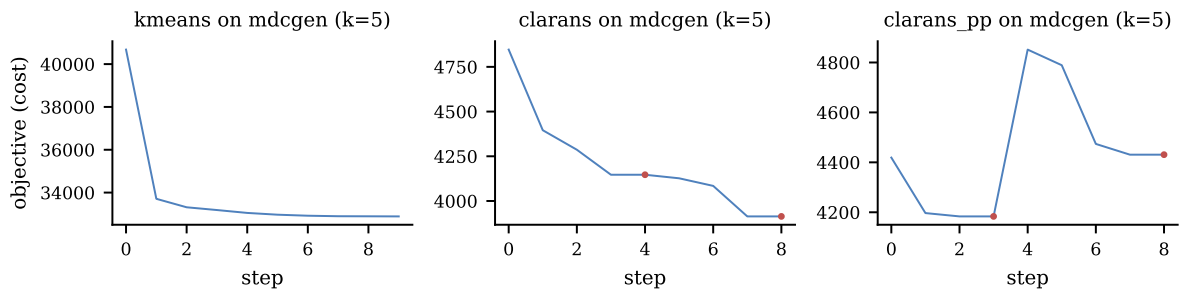


Figure 1: Trajectory records for one MDCGen task (n=500, d=10, k=5, 100 injected outliers, seed 1). k-means descends monotonically in a few EM steps; CLARANS and its k-means++-seeded variant take many local swap steps, and red dots mark rejected proposals.

lets an algorithm return a list of `Step` objects that are persisted as a Parquet trajectory table with one row per step:

$$(run_id, task, t,\ s_t,\ a_t,\ c_t,\ \Delta c_t = c_t - c_{t-1},\ accepted_t).$$

Here s_t is a numeric state snapshot, a_t is a typed action (e.g. `{"type": "swap", "out": 17, "in": 42}`), c_t is the scalar objective, and the boolean records whether a proposed action was accepted (local-search methods such as CLARANS [25] reject most proposals). The negative of Δc_t is a natural reward. Concatenated across runs, trajectories are a dataset for value models over $(s, a) \mapsto \Delta c$, for learned proposal distributions, and for imitation learning of clustering policies. The companion paper [8] trains such a policy. In the current release, 36 of the 47 benchmarked algorithms emit trajectories, and the in-distribution run stores 2,569 of them. Figure 1 shows three.

3.6 Algorithm cards, gap finder and mutator

Each registered algorithm has an *algorithm card*: its taxonomy family, a set of mechanism tags (e.g. `kpp_init`, `graph_laplacian`, `posterior_weighting`), a set of inductive biases (e.g. `convex_clusters`, `non_convex_capable`, `needs_k`), callables for time and memory complexity in (n, d, k), a hyperparameter space, and whether k is an input. Cards serve three purposes.

- **Theoretical predictions.** Every result row carries three predicted columns (`theoretical_*`: wall time, memory and an ARI upper bound) next to the measured values. After calibration, predicted wall time landed within 0.6–1.5× of measured for every algorithm. The deviations are diagnostic: mean shift's ratio of 0.59 shows that its card's $O(n^2)$ is pessimistic for the bin-seeded implementation.
- **Gap finding.** Given dataset fingerprints (effective dimension, convexity, outlier fraction, density skew and similar), the gap finder scores every card's predicted ARI upper bound and reports regimes where the best card falls below a threshold. Those regimes become new datasets.
- **Mutation.** The mutator applies four operators (`swap_mechanism`, `add_bias`, `crossover`, `stack`) to cards and keeps candidates whose predicted mean ARI beats the best existing card by at least 0.02. The top candidate in a 200-step random search was implemented and benchmarked [8].

The card predictions are an *ordering* heuristic, not a calibrated estimate. The mutator predicted 0.926 ARI for its proposed hybrid, which actually reached 0.747 (§6).

3.7 Dashboard and continuous evaluation

A static, dependency-free HTML dashboard served from the repository's `docs/` folder reads the results and trajectory JSON. It shows per-algorithm comparisons, a filterable per-task leaderboard, rank and Friedman tests computed in the browser, and a trajectory viewer. A build script converts any run directory into dashboard JSON, so refreshing the public leaderboard is one command plus a commit.

4 The benchmark suite

Datasets. Generators share one contract, DataSpec $\mapsto (X, y)$, with $y = -1$ for injected outliers and noise. They fall into six groups.

- *Convex and near-convex synthetic:* isotropic blobs, an MDCGen-style mixture [11] with outlier, noise and density knobs, sheared (anisotropic) blobs, imbalanced blobs, hierarchical nested Gaussians, and heavy-tailed Student-t mixtures.
- *Non-convex synthetic:* moons, concentric circles, spirals, Swiss roll, S-curve, rings and mixed shapes.
- *Heterogeneous density and contamination:* varying density and `extreme_outliers`, which uses a tunable outlier extremity.
- *Low rank in high dimension:* an inverse-PCA generator that places cluster means in a low-dimensional latent space and projects them to 50–200 ambient dimensions.
- *Real tabular data:* Iris [26], Wine, Breast Cancer, Digits (projected to 16 dimensions), Olivetti faces, Glass, Vehicle, Segment, Yeast and E. coli [27].
- *Other modalities:* time series (UCR ECG200 and Trace [28], plus a synthetic set) reduced to a 16-dimensional summary of 8 statistics, 4 FFT magnitudes and 4 autocorrelations; graph nodes (Zachary's karate club [29] and a stochastic block model [30]) reduced to 6 centralities and 10 Laplacian eigenvectors.
- *Adversarial:* `PowerLawStudentT` (anisotropic Student-t components with Zipf-distributed masses, whose first two moments look Gaussian) and `VariableDensityBridges` (dense cores with Cauchy tails joined by thin Poisson bridges, so no single ε works). Both were designed specifically to break the learned selectors.

Algorithms. The registry holds 50 implementations. The classical baselines are k-means with k-means++ seeding [31, 32], mini-batch k-means [7], MapReduce-style parallel k-means [6], BIRCH [33], Ward agglomerative [34], Gaussian mixtures via EM [35], spectral clustering [36], DBSCAN [37], OPTICS [38], mean shift [39], CLARANS [25], Chameleon [40], an unweighted consensus vote, PWCC [5] and S^{5}C [4]. The last four of the thesis's representatives (parallel k-means, mini-batch, PWCC and S^{5}C) cover its four taxonomy categories. The remaining entries are new algorithms described in the companion paper [8]: basis-mixture models, targeted fixes, synthesised pipelines, a graph-native method, ten learned selectors and a behaviour-cloned agent.

Evaluation splits.

- *In-distribution* (`benchmark.paper.demo.yaml`): 19 dataset configurations, seeds $\{1, 2, 3\}$, $n \leq 1000$; this is the split the learned selectors train on.
- *Held-out seeds* (`benchmark.holdout.yaml`): the same shapes with seeds $\{11, 12, 13\}$, which removes seed-level memorisation.
- *Out-of-distribution* (`benchmark.unseen.yaml`): seven configurations with (n, d, k) tuples, outlier extremities and seeds (100–102) absent from training, e.g. MDCGen at d=25, k=7, moons embedded in d=5, inverse-PCA at d=150, and outlier extremity 10.

5 Results

All numbers below come from the data committed in the repository (`docs/data` and `docs/data_unseen`). The figures are regenerated from those files by the script `make_figures.py` in `paper/scripts`.

5.1 Overall leaderboard

The in-distribution run covers 47 algorithms on 72 tasks (24 task variants from 19 dataset configurations, × 3 seeds), giving 3,354 result rows. Figure 2 shows average ARI ranks. Over the 64 tasks that every algorithm completed, the Friedman test gives $\chi^2 = 1188.8$ ($p = 7.1 \times 10^{-219}$). The top seven positions are learned selectors (§5.4 qualifies this). The strongest single algorithms are the graph-Laplacian methods: LMM (average rank 17.0, mean ARI 0.813) and spectral clustering (17.6, 0.809).

5.2 No algorithm dominates

Figure 3 breaks ARI down by dataset family for sixteen representative algorithms. Four populations emerge.

- *Convex-only* methods (k-means family, BIRCH, Ward, PWCC with k-means-style bases) score ≈ 1 on blobs and ≈ 0 on circles.
- *Graph-Laplacian* methods (spectral, LMM, AURA-v3) solve circles but reach only 0.52–0.61 on moons.
- *Graph-community* clustering (`louvain_knn`) is the only non-selector above 0.95 on moons, and scores 0.772 on the karate graph, where the next best non-selector reaches 0.506.
- *Gaussian mixtures* are best on Iris (0.904) and Wine (0.559), and within 0.01 of the best on Breast Cancer (0.794).

Every family has a different winner, which is the regime-mismatch problem that motivates algorithm selection.

5.3 Failure modes are few and recurrent

The algorithm analysis bundled with the repository diagnoses each algorithm along four axes: scaling, clean-data quality, outlier robustness and shape adaptability. Six failure modes account for almost every weakness: outlier sensitivity of mean-based updates, non-convex shape failure of Voronoi-cell methods, quadratic scaling, hyperparameter brittleness, local-minimum search, and wrong data assumptions. Some examples:

- With 100 outliers injected into 500 points, k-means loses 32% of its ARI and mean shift loses 59% (its bandwidth estimate is inflated by the outliers). GMM loses only 15%, because its posterior-weighted mean update gives outliers little influence.
- DBSCAN with a fixed ε=0.8 in d=10 returns one cluster on every task (ARI 0.00).
- S^5C scores 0.27 on isotropic blobs because it assumes a union of linear subspaces, but gains +0.36 on sheared data where that assumption holds.

Measured log–log wall-time slopes between n=200 and n=2000 range (excluding DBSCAN's degenerate one-cluster runs) from 0.33 (mini-batch k-means) through 1.01 (BIRCH, the only high-quality method that scales linearly in this range) to 1.78 (the MRI-inspired method).

Because the harness runs originals and variants side by side, it can test whether a diagnosed fix helps. Table 1 reports five one-line fixes. Replacing DBSCAN's constant ε with the k-distance knee of Ester et al. [37] lifts mean ARI by 0.59. One fix barely moved: trimming 10% of points per k-means iteration cannot absorb a 20% outlier rate. Two fixes introduced new failures. k-means++ seeding of CLARANS is biased *toward* outliers, because seeding probability is proportional to squared distance. The auto-ε knee is itself contaminated by outliers.

Table 1: Diagnosed one-line fixes, mean ARI over the 16-task demonstration sweep (round 2 snapshot).

original	fixed variant	ARI before	ARI after	Δ
`dbscan`	`dbscan_auto` (k-distance knee)	0.000	0.589	+0.589
`meanshift`	`meanshift_robust` (trimmed bandwidth)	0.470	0.622	+0.151
`clarans`	`clarans_pp` (k-means++ seeding)	0.581	0.656	+0.075
`pwcc`	`pwcc_diverse` (heterogeneous bases)	0.702	0.751	+0.049
`kmeans`	`kmeans_trimmed` (10% trim)	0.701	0.703	+0.003

5.4 Generalisation: the split that changes the conclusion

The learned selectors are trained on the benchmark's own results. Their in-distribution scores use task-level leave-one-out, which removes exact self-matches but still lets a sibling seed of the same configuration serve as a near neighbour. Clustbench therefore ships two stricter splits.

On the *held-out-seed* split, every algorithm scored slightly *higher* than in-distribution (the new seeds were easier), and the early selectors' gaps (-0.015, -0.020) matched those of non-learning methods (LMM -0.027). No seed-level memorisation was detectable.

The *out-of-distribution* split changes the picture (Figure 4). The four selectors that lead the in-distribution board lose between 0.115 and 0.128 ARI: v3, v5 and v7 fall from about 0.91 to 0.753 and v4 to 0.734. The simplest selector (v1, a k-nearest-neighbour vote over seven fingerprint features) *improves* to 0.840. LMM, which has no learned component, ranks first at 0.845, followed by v1, AURA-v2/v3 (0.837, 0.835) and `louvain_knn` (0.825). The more capacity a selector has to fit the training fingerprint distribution, the more it overfits it. The split that reveals this costs one configuration file, and we argue such a split should be a default part of any benchmark that also trains selectors on its own output.

5.5 The self-extending loop

Clustbench's learned selectors read the committed results file when they are imported, so adding an algorithm and re-running the benchmark retrains them with no code change. This has happened twice.

- A domain enlargement exposed the karate graph as a regime where every algorithm scored below 0.43 ARI. A graph-community algorithm, `louvain_knn`, was added and scored 0.772 there. On the next run, every fingerprint-based selector dispatched to it on that task, and the mean ARI of the best selectors rose by 0.029–0.038. No selector code changed between the two runs.
- Adding three discovery-agent algorithms, five datasets and two more selectors raised the stacker selector's mean ARI from 0.871 to 0.912, again without selector code changes.

The intelligence accumulates in the data, not the code.

6 Discussion

What the harness made cheap. Re-running every algorithm on every task is a single command. The research loop therefore became: diagnose a failure, ship a variant, re-rank against everything. The companion paper's fifteen rounds of algorithm development each cost one configuration change and one sweep. The latest round, in which LLM agents proposed, implemented, integrated and benchmarked three algorithms and five datasets, took about 17 minutes of wall clock.

Learning beats predicting from biases. Card-based predictions are useful for ordering candidate mutations, but their magnitudes are not calibrated. The mutator's top proposal was predicted at 0.926 ARI and measured at 0.747. Card-predicted upper bounds also *under*estimate how well classical methods generalise: spectral clustering's actual unseen ARI of 0.775 exceeds its card bound of 0.614. Selectors trained on measured results outperform selectors driven by inductive-bias arithmetic, but only within their training distribution.

Threats to validity.

- *Scale.* Most tasks have $n \leq 1000$. Wall-time slopes are measured only up to n=2000, so the constant-cost phases of some methods make their slopes look sub-linear. Clustbench's design does not limit n, but the reported sweeps were sized for continuous integration.
- *Seeds and statistics.* There are three seeds per configuration. The Friedman test is overpowered with 47 algorithms; post-hoc pairwise tests (e.g. Nemenyi) are needed before claiming that any two adjacent algorithms differ.
- *Timing.* Wall time and RSS deltas come from a single shared machine and include Python overhead; they support order-of-magnitude comparisons, not micro-benchmarks.
- *Selector leakage.* In-distribution selector scores are optimistic by construction; only the out-of-distribution split is an honest estimate.
- *Data provenance.* The UCR loader falls back to a synthetic generator if every mirror is unreachable. The inverse-PCA generator depends on a companion package and was disabled in the latest in-distribution run. Rows record their `dataset_source`.
- *Coverage.* Round-14 and later algorithms, and GMM, were not re-run on the out-of-distribution split.
- *Ground-truth outliers.* Injected outliers carry label -1 in the ground truth and are scored as their own class by ARI and NMI.

7 Conclusion

Clustbench turns "which clustering algorithm should I use?" into a reproducible, re-runnable measurement. It records full resource usage and the step-by-step optimisation trajectory, ships theoretical predictions next to measurements, and feeds its own output to selectors that improve when the registry grows. Its most useful result so far is a caution: selectors that lead the leaderboard on the distribution they were trained on can trail a classical graph-Laplacian mixture model on data they have not seen. Future work is larger-n sweeps, post-hoc pairwise testing, importing external dataset batteries such as Gagolewski's [9], and using the trajectory corpus to train learned components inside algorithms rather than selectors between them.

Availability. Code, configurations, committed results, the live dashboard (`https://ashail33.github.io/clustbench/`) and the scripts that regenerate every figure in this paper are at `https://github.com/Ashail33/clustbench`.

References

[1] Jon Kleinberg. An impossibility theorem for clustering. In *Advances in Neural Information Processing Systems*, volume 15, 2002.

[2] Iven Van Mechelen, Anne-Laure Boulesteix, Rainer Dangl, Nema Dean, Isabelle Guyon, Christian Hennig, Friedrich Leisch, and Douglas Steinley. Benchmarking in cluster analysis: A white paper. *arXiv preprint arXiv:1809.10496*, 2018.

[3] Ashail Maharaj. Review of big data clustering methods. MEng (structured) research assignment, Faculty of Engineering, Stellenbosch University, 2024. Supervisor: A.P. Engelbrecht.

[4] Shin Matsushima and Maria Brbić. Selective sampling-based scalable sparse subspace clustering. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[5] Rasim M. Alguliyev, Ramiz M. Aliguliyev, and Lyudmila V. Sukhostat. Weighted consensus clustering and its application to big data. *Expert Systems with Applications*, 150:113294, 2020.

[6] Weizhong Zhao, Huifang Ma, and Qing He. Parallel k-means clustering based on MapReduce. In *Cloud Computing (CloudCom 2009)*, volume 5931 of *Lecture Notes in Computer Science*, pages 674–679, 2009.

[7] D. Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web*, pages 1177–1178, 2010.

[8] Ashail Maharaj. Synthesising clustering algorithms from benchmark failure modes: Basis mixtures, region-adaptive pipelines, learned routers and a behaviour-cloned clustering agent, 2026. Companion paper, https://github.com/Ashail33/clustbench/tree/master/paper/algorithms.

[9] Marek Gagolewski. A framework for benchmarking clustering algorithms. *SoftwareX*, 20: 101270, 2022.

[10] Adil Fahad, Najlaa Alshatri, Zahir Tari, Abdullah Alamri, Ibrahim Khalil, Albert Y. Zomaya, Sebti Foufou, and Abdelaziz Bouras. A survey of clustering algorithms for big data: Taxonomy and empirical analysis. *IEEE Transactions on Emerging Topics in Computing*, 2 (3):267–279, 2014.

[11] Félix Iglesias, Tanja Zseby, Daniel Ferreira, and Arthur Zimek. MDCGen: Multidimensional dataset generator for clustering. *Journal of Classification*, 36:599–618, 2019.

[12] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[13] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.

[14] John R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

[15] Pavel Brazdil, Christophe Giraud-Carrier, Carlos Soares, and Ricardo Vilalta. *Metalearning: Applications to Data Mining*. Springer, 2009.

[16] Bernhard Pfahringer, Hilan Bensusan, and Christophe Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning*, pages 743–750, 2000.

[17] Marcilio C. P. de Souto, Ricardo B. C. Prudêncio, Rodrigo G. F. Soares, Daniel S. A. de Araujo, Ivan G. Costa, Teresa B. Ludermir, and Alexander Schliep. Ranking and selecting clustering algorithms using a meta-learning approach. In *2008 IEEE International Joint Conference on Neural Networks*, pages 3729–3735, 2008.

[18] Daniel G. Ferrari and Leandro Nunes de Castro. Clustering algorithm selection by meta-learning systems: A new distance-based problem characterization and ranking combination methods. *Information Sciences*, 301:181–194, 2015.

[19] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[20] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1): 193–218, 1985.

[21] Alexander Strehl and Joydeep Ghosh. Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.

[22] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[23] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

[24] J. C. Dunn. A fuzzy relative of the ISODATA process and its use in detecting compact well-separated clusters. *Journal of Cybernetics*, 3(3):32–57, 1973.

[25] Raymond T. Ng and Jiawei Han. CLARANS: A method for clustering objects for spatial data mining. *IEEE Transactions on Knowledge and Data Engineering*, 14(5):1003–1016, 2002.

[26] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[27] Dheeru Dua and Casey Graff. UCI machine learning repository. University of California, Irvine, School of Information and Computer Sciences, 2019.

[28] Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

[29] Wayne W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4):452–473, 1977.

[30] Paul W. Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social Networks*, 5(2):109–137, 1983.

[31] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[32] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[33] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pages 103–114, 1996.

[34] Joe H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963.

[35] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.

[36] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*, volume 14, 2002.

[37] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

[38] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. OPTICS: Ordering points to identify the clustering structure. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, pages 49–60, 1999.

[39] Dorin Comaniciu and Peter Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002.

[40] George Karypis, Eui-Hong Han, and Vipin Kumar. Chameleon: Hierarchical clustering using dynamic modeling. *Computer*, 32(8):68–75, 1999.

A Record schema

field	description
`algo`, `dataset_id`	algorithm and dataset keys
`n_samples`, `n_features`, `k_target`	task shape
`compactness`, `outliers`, `noise`, `density`, `seed`	generator knobs
`wall_time_s`, `rss_delta_mb`	wall-clock time, memory delta
`cpu_user_s`, `cpu_system_s`, `read_bytes`, `write_bytes`	CPU and I/O
`ari`, `nmi`	external validity
`silhouette`, `davies_bouldin`, `dunn`, `compactness`, `separation`	internal validity
`n_clusters_found`, `extra`	output cardinality, algorithm extras
`n_steps`, `trajectory_path`	trajectory length and Parquet path
`theoretical_*`	card-based predictions (wall time, RSS, ARI bound)

Trajectory rows add `run_id`, `step_idx`, `cost`, `delta_cost`, `accepted`, `action` and `state`.

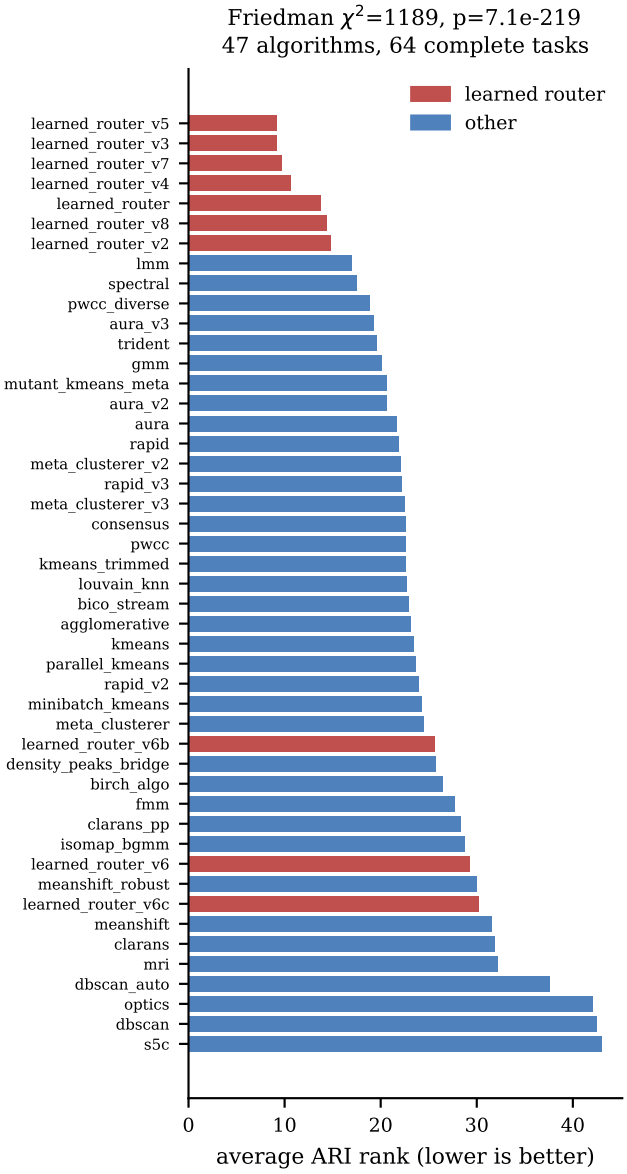


Figure 2: Average ARI rank over the in-distribution split. Red bars are learned selectors, which are trained on this split (leave-one-out at the task level).

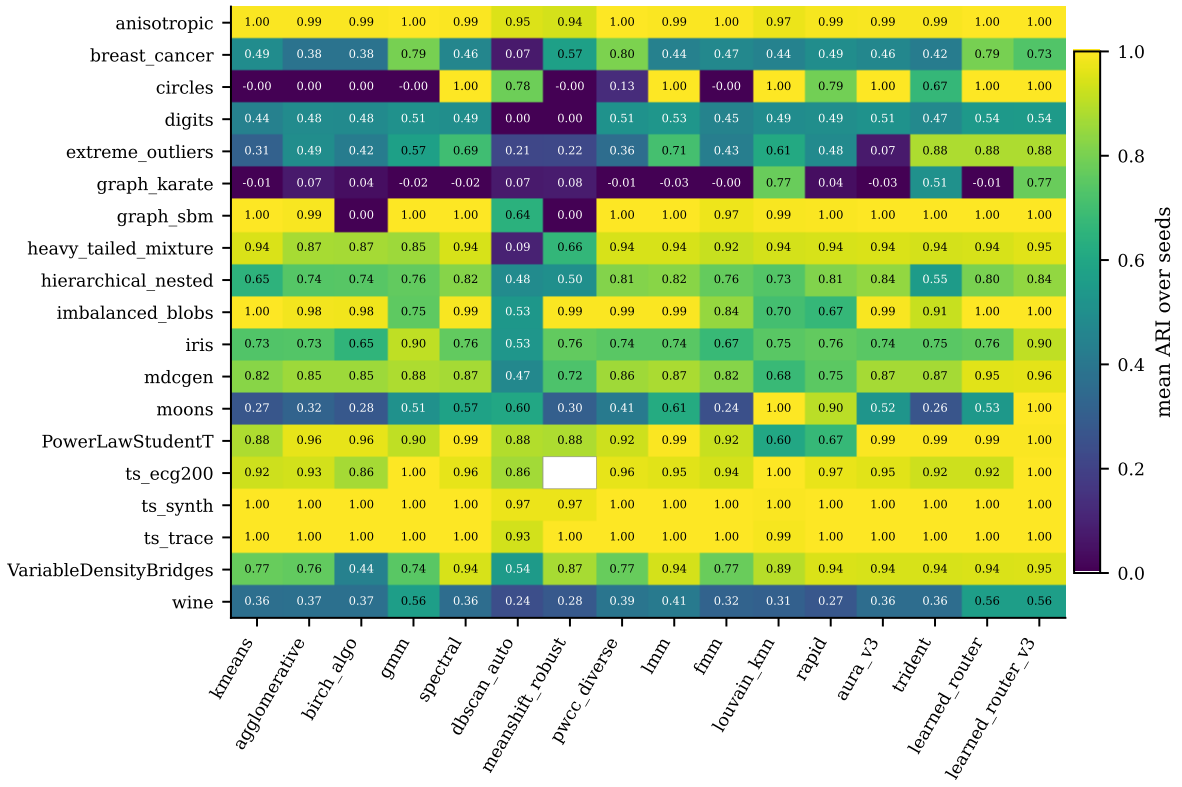


Figure 3: Mean ARI over seeds for each dataset family (rows) and representative algorithm (columns), in-distribution split. The blank cell is a run that failed and was logged.

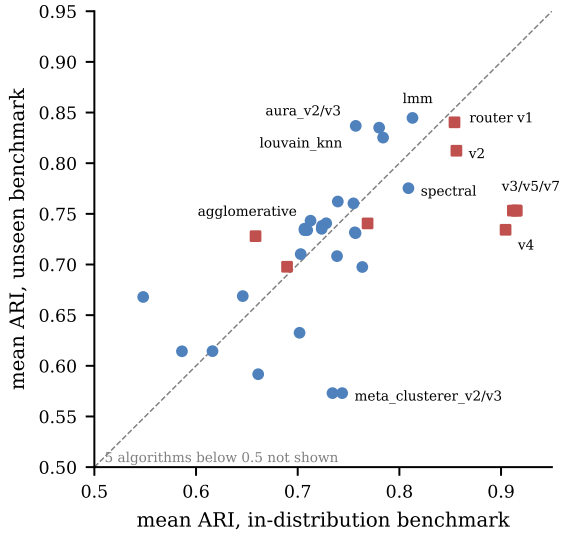


Figure 4: Mean ARI in-distribution vs. on the out-of-distribution split for the 42 algorithms run on both. Squares are learned selectors. Points below the diagonal lost accuracy off-distribution; the highest-ranked selectors lose the most.